
Time Complexity: Introduction

Md. Owaliur Rahman

BSc in CSE, BUET

Lecturer, CSE, Presidency University

September 23, 2026

1. Simple Linear Loop

```
1  for (int i = 0; i < n; i++) {  
2      printf("%d\n", i);  
3  }
```

- **Runtime function:** $T(n) = n$
the loop body executes once for every value of i from 0 to $n - 1$.
- **Time complexity:** n — **Linear**. Work grows in direct proportion to n .

2. Linear Loop (Step Size 2)

```
1  for (int i = 0; i < n; i += 2) {  
2      printf("%d\n", i);  
3  }
```

- **Runtime function:** $T(n) = \frac{n}{2}$
 i skips every other value, so the body runs about half as many times.
- **Time complexity:** n — **Linear**. Constant factors ($\frac{1}{2}$) are dropped; growth is still proportional to n .

3. Halving Loop

```
1 for (int i = n; i > 0; i /= 2) {  
2     printf("%d\n", i);  
3 }
```

- **Runtime function:** $T(n) = \log_2 n$
 i is divided by 2 each iteration, so the loop runs only as many times as n can be halved before reaching 0.
- **Time complexity:** $\log n$ — **Logarithmic**. Very slow growth; doubling n adds just one more iteration.

4. Nested Loop (2D)

```
1  for (int i = 0; i < n; i++) {  
2      for (int j = 0; j < n; j++) {  
3          printf("%d %d\n", i, j);  
4      }  
5  }
```

- **Runtime function:** $T(n) = n \times n = n^2$ — for each of the n outer iterations, the inner loop runs n times.
- **Time complexity:** n^2 — **Quadratic**. Two independent loops over n multiply their costs.

5. Triple Nested Loop

```
1  for (int i = 0; i < n; i++)
2      for (int j = 0; j < n; j++)
3          for (int k = 0; k < n; k++) {
4              s = i + j + k;
5              printf("%d %d %d\n", s);
6          }
```

- **Runtime function:** $T(n) = n \times n \times n = n^3$ — three fully nested loops, each running n times independently of the others.
- **Time complexity:** n^3 — **Cubic**. One more nested loop than the quadratic case, one more power of n .

6. Loop with a Halving Inner Loop

```
1  for (int i = 0; i < n; i++) {  
2      int x = n;  
3      while (x > 1) {  
4          x = x / 2;  
5      }  
6  }
```

- **Runtime function:** $T(n) = n \times \log_2 n$ — the outer loop runs n times, and for *each* of those runs the inner while halves x from n to 1, costing $\log_2 n$ steps.
- **Time complexity:** $n \log n$ — **Linearithmic**. A linear loop containing a logarithmic loop.

7. Fibonacci (Naive Recursion)

```
1  int fibonacci(int n) {  
2      if (n == 0) return 0;  
3      if (n == 1) return 1;  
4      return fibonacci(n - 1) + fibonacci(n - 2);  
5  }
```

- **Runtime function:** 2^n because every call (except the base cases) branches into two smaller calls.
- **Time complexity:** 2^n — Exponential.

8. Triple Recursive Calls

```
1 void f(int n) {  
2     if (n == 0) return;  
3     f(n - 1);  
4     f(n - 1);  
5     f(n - 1);  
6 }
```

- **Runtime function:** 3^n because each call spawns *three* calls on a problem one smaller.
- **Time complexity:** 3^n — **Exponential**. Branching factor 3 instead of 2 grows even faster than Fibonacci.

9. Generating Permutations

```
1 void permute(int arr[], int n) {  
2     if (n == 1) return;  
3     for (int i = 0; i < n; i++) {  
4         permute(arr, n - 1);  
5     }  
6 }
```

- **Runtime function:** $T(n) = n \times T(n - 1)$ — at every level of recursion, the loop makes n calls to a problem of size $n - 1$, giving $n \times (n - 1) \times (n - 2) \times \dots \times 1$ total calls.
- **Time complexity:** $n!$ — **Factorial**. Characteristic cost of generating all permutations of n items.

10. Constant-Time Function

```
1 int sum(int a, int b) {  
2     return a + b;  
3 }
```

- **Runtime function:** $T(n) = 1$
a single addition and return, independent of any input size n .
- **Time complexity:** 1 — **Constant**. No loop or recursion; cost never changes.

11. Constant-Time Function (Multiple Ops)

```
1  int triple(int a) {  
2      int out = a;  
3      out += a;  
4      out += a;  
5      return out;  
6  }
```

- **Runtime function:** $T(n) = 4$
a fixed, small number of operations (four statements) that never depends on an input size n .
- **Time complexity:** 1 — **Constant**. More statements than Example 10, but still a fixed count.

Growth rates at a glance

Growth	Simplest pattern	Complexity
Constant	Fixed number of operations (e.g. <code>arr[0]</code>)	1
Logarithmic	Halve the problem each step (e.g. <code>i /= 2</code>)	$\log n$
Linear	Single loop over n elements	n
Linearithmic	Nested $\log n$ loop inside an n loop	$n \log n$
Quadratic	Two nested loops over n	n^2
Cubic	Three nested loops over n	n^3
Exponential	Two recursive calls per level	2^n
Factorial	Generate all permutations of n items	$n!$

12. Matrix Multiplication

```
1  #define R1 2 // number of rows in Matrix-1
2  #define C1 2 // number of columns in Matrix-1
3  #define R2 2 // number of rows in Matrix-2
4  #define C2 3 // number of columns in Matrix-2
5
6  void multiplyMatrix(int m1[][C1], int m2[][C2])
7  {
8      int result[R1][C2];
9
10     for (int i = 0; i < R1; i++) {
11         for (int j = 0; j < C2; j++) {
12             result[i][j] = 0;
13
14             for (int k = 0; k < R2; k++)
15                 result[i][j] += m1[i][k] * m2[k][j];
16
17             printf("%d\t", result[i][j]);
18         }
19
20         printf("\n");
21     }
22 }
```

- **Runtime:** $T = R1 \times C2 \times R2$; for square $n \times n$ matrices, $T(n) = n^3$.
- **Complexity:** n^3 (or $R1 \cdot C2 \cdot R2$) — **Cubic**.

13. Binary Search

```
1  int binarySearch(int arr[], int n, int x) {
2      int low = 0;
3      int high = n-1;
4      while (low <= high) {
5          int mid = low + (high - low) / 2;
6          if (arr[mid] == x) return mid;
7          if (arr[mid] < x) low = mid + 1;
8          else high = mid - 1;
9      }
10     return -1;
11 }
```

- **Runtime function:** Each comparison discards half the remaining search space, so at most $\log_2 n$ iterations run.
- **Time complexity:** $\log n$ — **Logarithmic**

Order of Growth

$$\begin{array}{ccccccccccc} 1 & < & \log n & < & n & < & n \log n & < & n^2 & < & n^3 & < & 2^n & < & n! \\ \text{Constant} & & \text{Logarithmic} & & \text{Linear} & & \text{Linearithmic} & & \text{Quadratic} & & \text{Cubic} & & \text{Exponential} & & \text{Factorial} \end{array}$$

Exam Practice I

Analyze the time and space complexity of the following function:

```
1 void analyze(int arr[], int n)
2 {
3     int sum = 0;
4
5     // Part 1
6     for (int i = 0; i < n; i++)
7         sum += arr[i];
8
9     // Part 2
10    for (int i = 0; i < n; i++) {
11        int j = 1;
12
13        while (j < n)
14            j *= 2;
15
16        for (int k = 0; k < n; k++)
17            printf("%d ", arr[i]);
18    }
19
20    // Part 3
21    int *temp = malloc(n * sizeof(int));
22
23    for (int i = 0; i < n; i++)
24        temp[i] = arr[i];
25}
```

Exam Practice II

```
26 // Part 4
27 for (int i = 0; i < n; i++) {
28     for (int j = i + 1; j < n; j++) {
29         if (arr[i] > arr[j]) {
30             int x = arr[i];
31             arr[i] = arr[j];
32             arr[j] = x;
33         }
34     }
35 }
36
37 free(temp);
38 }
```

Key Takeaways

Time Complexity

- Count how operations **grow with** n , not exact totals
 - Loops $\rightarrow$ linear; nested loops $\rightarrow$ multiply
 - Halving each step (e.g. $i /= 2$) $\rightarrow \log n$
 - Recursion: branches $\times$ depth $\rightarrow$ total calls
-

Thank you

Questions welcome